Лабораторийн ажил №11. TicTacToe буюу X/0 тоглоом клиент/сервер хувилбар

Агуулга

Tic-Tac-Toe (Монголчуудын нэрлэж заншсанаар X/O) нь хоёр тоглогч ээлжлэн нүүдэг логик тоглоом юм. Ерөнхийдөө 3x3 хэмжээтэй хүснэгтэд тоглодог боловч хүснэгтийн хэмжээг илүү том байхаар шийдэж болно. Тоглогчид хоосон нүднүүдийг ээлжлэн өөрийн тэмдгээр (X эсвэл O) бөглөх бөгөөд хэвтээ, босоо эсвэл диагональ чиглэлд гурван тэмдгийг түрүүлж дараалуулсан нь ялагч болдог.

Энэхүү систем нь хоёр тоглогч нэг компьютер дээр хамт сууж тоглох боломжтой хувилбар юм.

Технологийн шийдлийн хувьд:

- Backend: Java хэл дээр суурилсан NanoHTTPD сервер
- Frontend: React.js болон TypeScript (TS)

Энэхүү клиент-сервер (Client-Server) архитектур нь веб программ хангамж боловсруулахад өргөн ашиглагддаг түгээмэл загвар юм. Хэдийгээр энд Java болон React-ийг хослуулсан боловч ерөнхийдөө backend болон frontend-ийн технологиудыг хэрхэн хослуулах нь маш өргөн сонголттой байдаг.

Хүлээлгэн өгөх зүйлс

- Тоглоомын "instructions" хэсэгт одоогийн нүүдэл хийж буй тоглогч болон тоглоомын ялагчийг дэлгэцэнд харуулдаг байхаар функционалыг өргөжүүлэн хөгжүүлэх
- Хэрэглэгч хийсэн нүүдлээ буцаах (Undo) боломжийг олгосон нэмэлт хөгжүүлэлт хийх
- Дизайн болон харагдах байдлыг сайжруулах: CSS файл ашиглан дэлгэцийн өнгө төрх, интерфейсийн харагдах байдлыг сайжруулан өөрчлөх
- Дээрх ажлуудыг гүйцэтгэж, өөрийн Git репозиторид (repository) байршуулан, холбоос (link)-ыг илгээх

Заавар

https://github.com/gobi/f.csa311.25b-lab11 репог форк хийж, clone хийж авах

Тохируулах зааварчилгааг төслийн README хэсгээс авна уу. Программыг ажиллуулсны дараа http://localhost:3000/ холбоосоос доорх хэсэгчилсэн зурагт үзүүлсэн шиг GUI-ээр дамжуулан тоглоомоо тоглох боломжтой болно.

Энэ удаагийн лаб бол энэ төрлийн кодыг уншиж, удирдаж сурах тухай юм. Даалгаврын хувьд их цаг шаардахгүй хэд, хэдэн энгийн сайжруулалт хийнэ. Тиймээс кодыг уншиж, мэдээллийн урсгал, хяналтын талаар ойлгохыг урьтал болго.

Заавар:

Одоогийн байдлаар тоглоом хэрэглэгчдэд хангалттай мэдээлэл өгөхгүй байна. Жишээлбэл, хэн нэгэн ялахад тоглоом зүгээр л ямар ч үйлдэлгүй (input авахгүй) зогсдог. Үүнийг засахын тулд HTML-ийн үндсэн хэсэгт `instructions` ID-тай зааварчилгааны элемент нэмэх шаардлагатай (үүний CSS загвар нь бэлэн байгаа). Энэхүү элемент нь хэний ээлж болох, эсвэл тоглоомд хэн ялсныг тодорхой харуулдаг байх ёстой.

> Зөвлөмж: Клиент (Frontend/View) болон Сервер (Backend/Controller) тал дээр энэхүү өөрчлөлтөд өртөх бүх хэсгүүдийг тооцож үзээрэй. Бусад элементүүд хэрхэн шинэчлэгдэж байгааг жишээ болгон ашиглаарай.

Одоогоор "Undo" товч ямар нэгэн үйлдэл гүйцэтгэхгүй байгаа. Өмнөх дасгалуудтай адил, өөрчлөлт орох шаардлагатай бүх хэсгийг (Frontend-ээс Backend хүртэл) дахин хянаж үзэх хэрэгтэй. Үүний тулд тоглоомын логик хэсэгт нэвтэрч, нүүдлийн түүхийг хянах замаар буцаах үйлдлийг хэрэгжүүлнэ.

> Хэрэгжүүлэлт: Тоглоомын явцын түүхийг хадгалах "History" функцийг ашиглан олон үйлдлийг дараалан буцаах (Multi-step Undo) боломжтой болгоно уу. Хэрэглэгч товч дарах үед хүсэлт Backend рүү хэрхэн очиж, хариу нь дэлгэцэнд хэрхэн тусах урсгалыг бүрэн зураглаарай.

Тоглоомын харагдах байдлыг илүү үзэмжтэй болгох хэрэгтэй. Өгөгдсөн CSS-ийг өргөжүүлэх, эсвэл тусад нь файл болгон салгаж, илүү боловсронгуй загвар оруулах боломжтой.

Зөвлөмж: Түлхүүр файлууд

Tic-Tac-Toe тоглоомын Frontend болон Backend-ийн хэрэгжүүлэлтийг ойлгохын тулд дараах 4 гол файлтай ажиллах шаардлагатай. Үүнд:

- App.tsx: Энэхүү React төслийн үндсэн бүрэлдэхүүн хэсэг (Root Component). Программын ерөнхий бүтэц, төлөв (state) болон логик энд төвлөрнө.
- App.css: Программын харагдах байдал, хэв загварыг тодорхойлох үндсэн CSS файл.
- Cell.tsx: Тоглоомын самбарын нэг нүдийг дүрслэх React компонент (Component). Хэрэв шаардлагатай бол энэхүү компонентдоо зориулж тусдаа загварын файл үүсгэж ашиглаж болно.
- game.ts: Тоглоомын өгөгдлийн бүтцийг (`GameState`, `Cell` гэх мэт) тодорхойлох файл. Эдгээр интерфэйс болон төрлүүд (Types) нь Backend талд тодорхойлсон өгөгдлийн бүтэцтэй яг таарч (matching) байх ёстойг анхаарна уу.

Зөвлөмж: Frontend-ийг Backend-тэй холбох

Энэхүү программ нь Frontend/Backend архитектуртай бөгөөд дараах зарчмаар ажиллана:

- Java Backend: NanoHTTPD ашиглан http://localhost:8080/ хаяг дээр сервер ажиллуулна. Энэхүү сервер нь хоёр үндсэн замаар (endpoint) хүсэлт хүлээн авдаг:
  - `/newgame`: Шинэ тоглоом эхлүүлэх

  - `/play`: Тодорхой нүүдэл хийх үйлдлийг боловсруулах
- React Frontend (TypeScript): Хэрэглэгчийн интерфейс нь http://localhost:3000/ хаяг дээр ажиллах бөгөөд тоглоомын үйлдэл бүрд (жишээ нь: /newgame эсвэл /play замууд руу хандах үед) шаардлагатай мэдээллийг NanoHTTPD сервер рүү хүсэлт (Request) болгон илгээнэ.
- Өгөгдлийн солилцоо: Сервер нь хүсэлтийг хүлээн авч боловсруулаад, шинэчлэгдсэн тоглоомын төлөв болон нүднүүдийн мэдээллийг JSON хэлбэрээр React руу хариу (Response) болгон илгээнэ. React нь хүлээн авсан өгөгдлөөрөө дэлгэцийг дахин зурж (Render), хэрэглэгчид шинэчлэгдсэн төлөвийг харуулдаг.

HTML/CSS тойм

HTML (HyperText Markup Language) нь вэб хуудас бүтээх стандарт тэмдэглэгээний хэл юм. Бүтцийн хувьд XML-тэй төстэй. Харин CSS (Cascading Style Sheets) нь HTML баримтын загвар, дизайныг тодорхойлоход ашиглагддаг. Өөрөөр хэлбэл, CSS нь HTML элементүүд дэлгэц дээр хэрхэн харагдахыг (өнгө, хэмжээ, байршил г.м) зохицуулдаг. Хэрэв та эдгээр технологиудтай анх удаа танилцаж байгаа бол дараах онлайн эх сурвалжуудыг ашиглахыг зөвлөж байна. Үүнд:

- HTML хичээл: w3schools.com/html
- CSS хичээл: w3schools.com/css

Nanohttpd тойм

NanoHTTPD бол энгийн веб аппликейшн хөгжүүлэлтэд зориулагдсан, авсаархан (lightweight) Java сервер юм. Түүний гол давуу тал нь маш бага хэмжээтэй бөгөөд системийн ажиллагаанд нэмэлт ачаалал, хоцрогдол үүсгэдэггүй тул бэлэн байгаа программд суулгаж ашиглахад нэн тохиромжтой.

Энэхүү Tic-Tac-Toe тоглоомын арын алба буюу Backend сервер нь NanoHTTPD дээр суурилсан. Техникийн дэлгэрэнгүй мэдээллийг NanoHTTPD зааварчилгаа холбоосоор орж үзнэ үү. (https://www.baeldung.com/nanohttpd).

React.js тойм

React.js нь интерактив, динамик хэрэглэгчийн интерфейс (UI) бүтээхэд зориулагдсан JavaScript сан (Library) юм. Үүнийг TypeScript-тэй хослуулан ашиглах бүрэн боломжтой. React-ийн үндсэн зарчим нь "Компонент" (Component) гэж нэрлэгдэх жижиг, бие даасан кодын хэсгүүдийг ашиглан нарийн төвөгтэй интерфейсийг хялбар аргаар угсрахад оршдог.

Бид энэхүү төсөлд React.js-ийн хамгийн анхан шатны функцуудыг ашиглан тоглоомын фронтенд хэсгийг хөгжүүлсэн.

Бяцхан түүх, яагаад React.js гэж?

1-р үе шат: Статик вэб (Static Web)

Интернетийн эхэн үед ихэнх веб хуудсууд статик шинжтэй байв. Инженерүүд контентоо зөвхөн HTML ашиглан тодорхойлдог байсан бөгөөд мэдээллийг өөрчлөхийн тулд серверийн

арын хэсэгт байрлах HTML файлыг гараар шинэчлэх шаардлагатай болдог байлаа. Эдгээр веб хуудсууд нь хэрэглэгчтэй харилцах боломж (interaction) хязгаарлагдмал төдийгүй, хөтөч (browser) дээр нэгэнт ачаалсны дараа өгөгдлийг динамикаар дахин ачаалах боломжгүй байв.

2-р үе шат: Динамик веб ба JavaScript (Client-side Scripting)

Хөтчүүд JavaScript-ийг дэмждэг болсноор веб хөгжүүлэлтэд томоохон өөрчлөлт гарсан. Инженерүүд клиент тал (хэрэглэгчийн хөтөч) дээр веб хуудсыг өөрчлөх, хөдөлгөөнт дүрс нэмэх, өгөгдлийг динамикаар ачаалах боломжтой болов. Жишээ нь: Хэрэглэгч хуудсыг доош гүйлгэх (scroll) үед шинэ мэдээлэл өөрөө нэмэгдэж ачаалагддаг вебсайтууд энэ технологи дээр суурилдаг.

3-р үе шат: Орчин үеийн фреймворкууд ба React.js

Өнөөг хүртэл ихэнх вебсайтууд 2-р шатны технологид тулгуурласаар ирсэн. Гэвч хуучны JavaScript сангууд (jQuery гэх мэт) нь өгөгдлийг гараар ачаалж, HTML элементийг нэг бүрчлэн нэмэх, устгахыг шаарддаг байв. Веб хуудас өндөр интерактив, динамик болох тусам энэ арга барил нь хэтэрхий нүсэр бөгөөд алдаа гарах магадлал өндөртэй болсон юм.

Үүнийг шийдвэрлэхийн тулд React.js зэрэг орчин үеийн фреймворкууд гарч ирсэн. React-ийн тусламжтайгаар та өөрийн өгөгдөл (Data/State) болон кодыг HTML элементүүдтэй хялбархан холбож болох бөгөөд өгөгдөл өөрчлөгдөх үед фреймворк нь дэлгэцийн харагдацыг (UI) автоматаар шинэчилдэг болсон юм.

Components

Компонент нь React-ийн үндсэн суурь элемент юм. Компонент бүр дараах гурван зүйлийг өөртөө агуулдаг. Үүнд:

- Бүтэц (HTML): Өгөгдлийг дэлгэцэнд хэрхэн дүрслэхийг тодорхойлох HTML элементүүд
- Өгөгдөл (Data): Тухайн компоненттой холбогдсон өгөгдөл. Жишээ нь: Тоглоомын самбарыг харуулах компонент нь самбарын утгуудыг агуулсан өгөгдөлтэй холбогдсон байна.
- Логик (Logic): Өгөгдөл болон харагдац хэрхэн динамикаар өөрчлөгдөх, хэрэглэгчийн үйлдэлд (товчлуур дарах гэх мэт) хэрхэн хариу үйлдэл үзүүлэхийг зохицуулах код

Иймд бид веб аппликейшнийг жижиг хэсгүүдэд (компонентуудад) хуваах, хэсэг бүрийг өөрийн гэсэн өгөгдөл, логик, харагдацтайгаар бие даасан байдлаар хөгжүүлэх боломжтой.

Маш энгийн component нь дараах байдлаар харагдаж болно.

```
class Welcome extends React.Component {
  constructor(props) {
    super(props);
    this.state = { };
  }

  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

Props болон State-ийн ялгаа

React компонент бүрт дараах хоёр гол объект байдаг. Үүнд:

- Props (Properties): Энэ нь эцэг (parent) компонентоос дамжуулсан өгөгдөл юм. Жишээ нь: `Welcome` компонентод name гэсэн утгыг дамжуулбал бид `this.props.name` ашиглан тэрхүү нэрэнд хандана. Props-ийг хүлээн авсан компонент өөрөө өөрчлөх ёсгүй, зөвхөн дамжуулсан эцэг компонент нь өөрчлөх эрхтэй.
- State (Төлөв): Энэ нь компонентын дотоод өгөгдөл бөгөөд тухайн компонент өөрөө удирддаг. State-ийг шинэчлэхийн тулд `setState()` функцийг ашиглана. Утга шинэчлэгдэх бүрд React дэлгэцийг (UI) автоматаар дахин зурдаг.
- `render()` функц: Компонент хэрхэн харагдахыг тодорхойлдог цорын ганц заавал байх ёстой функц юм.

JSX-ийн тухай

React дээр бид JSX (JavaScript XML) ашиглан HTML-ийг JavaScript эсвэл TypeScript код дотор шууд бичдэг. Энэ нь кодыг илүү ойлгомжтой, динамик болгодог. Жишээ нь, таг (tag) дотор `{this.props.name}` гэж бичвэл нэр өөрчлөгдөх үед React харагдах байдлыг автоматаар шинэчлэх болно.

Зарим чухал тэмдэглэл:

- Props: Компонент дотроос өөрчлөх боломжгүй (Read-only)
- State: Компонент дотор өөрчлөх боломжтой (Mutable via setState)
- Constructor: Заавал байх шаардлагагүй, харин render() функц заавал байх ёстой.

Прокси (proxy) тохиргоо

Энэхүү лабораторийн ажилд ашиглаж буй нэг чухал техник бол Proxy тохиргоо юм. Прокси нь үндсэндээ фронтенд (Frontend) талаас гарч буй бүх харьцангуй замын хүсэлтүүдийг package.json файлд зааж өгсөн үндсэн URL (Base URL) руу чиглүүлэх үүргийг гүйцэтгэдэг.

Ажиллах зарчим:

Энэхүү даалгаврын хүрээнд проксиг http://localhost:8080/ (Backend сервер ажиллаж буй хаяг) руу чиглүүлсэн байгаа. Ингэснээр кодонд `await fetch("/newgame")` гэж бичих үед хөтөч үүнийг автоматаар http://localhost:8080/newgame руу илгээх GET хүсэлт гэж ойлгоно.

```
{} package.json > ...
1  {
2    "name": "front-end",
3    "version": "0.1.0",
4    "private": true,
5    "proxy": "http://localhost:8080",
```

JSON

localhost:8080/newgame

localhost:8080/newgame

{ "cells": [{ "text": "", "playable": true, "x": 0, "y": 0 } , { "text": "", "playable": true, "x": 1, "y": 0 } , { "text": "", "playable": true, "x": 2, "y": 0 } , { "text": "", "playable": true, "x": 0, "y": 1 } , { "text": "", "playable": true, "x": 1, "y": 1 } , { "text": "", "playable": true, "x": 2, "y": 1 } , { "text": "", "playable": true, "x": 0, "y": 2 } , { "text": "", "playable": true, "x": 1, "y": 2 } , { "text": "", "playable": true, "x": 2, "y": 2 }]}

JSON нь өгөгдөл солилцох, хадгалахад зориулагдсан хөнгөн бөгөөд хүн уншихад хялбар текстэн формат юм. Энэ нь программчлалын хэлнүүдэд байдаг "Dictionary" эсвэл "Map" (Түлхүүр-Утга хослол) бүхий объекттой ижил бүтэцтэй.

JSON-ийн гол онцлог:

- Түлхүүр ба Утга: Мэдээлэл бүр нь түлхүүр (Key) болон түүнд харгалзах утга (Value)-аас бүрдэнэ. Түлхүүр нь үргэлж хашилтад ("") бичигддэг.
- Түгээмэл хэрэглээ: Клиент (React) болон Сервер (Java) хооронд өгөгдөл дамжуулахад стандарт формат болон ашиглагддаг.
- Уншихад хялбар: Бүтэц нь маш цэгцтэй тул хүн болон машин аль аль нь уншиж, боловсруулахад хялбар байдаг.

```
{
  "cells": [
    {
      "text": "",
      "playable": true,
      "x": 0,
      "y": 0
    },
    {
      "text": "",
      "playable": true,
      "x": 1,
      "y": 0
    }
  ]
}
```

Энэ нь backend-ээс frontend руу илгээсэн мэдээллийн формат юм.